

EECS 489 - WN 25

Discussion 10

Ethernet, DHCP, Wireless Networks, More Project 4

Announcements

Assignment 4 has been released! Due on **Tuesday, April 22.**

You have 3 group late days in total for Assignment 2-4.

Agenda

- Lecture Exercises: Ethernet, DHCP, Wireless Networks
- More Project 4!
- More Modern C++ (smart pointers, casting)

Lecture Exercises

Q1 Getting Connected

Suppose you walk into a room, open your laptop, connect to Ethernet, and navigate to <https://cse.engin.umich.edu/research> in your favorite browser.

What are all the network protocol steps that take place, starting from powering on your laptop to getting the web page?

Q1 Getting Connected

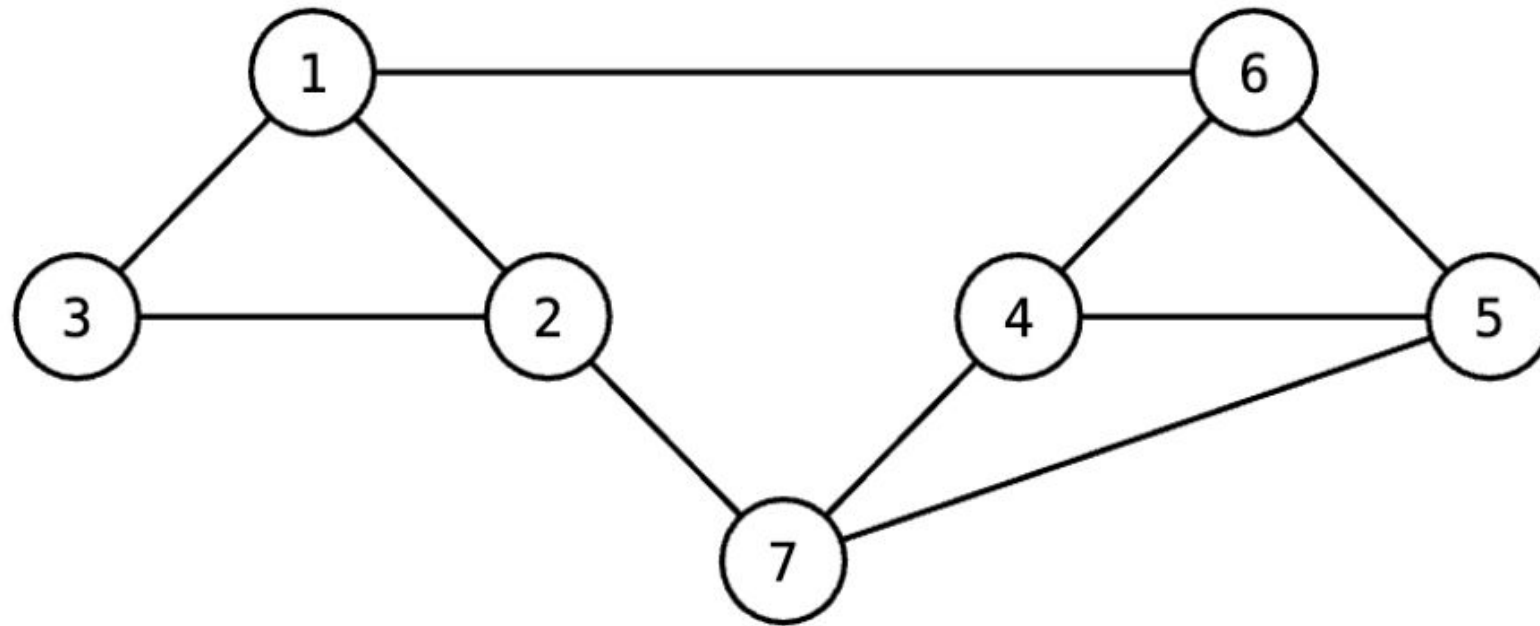
Suppose you walk into a room, open your laptop, connect to Ethernet, and navigate to <https://cse.engin.umich.edu/research> in your favorite browser.

What are all the network protocol steps that take place, starting from powering on your laptop to getting the web page?

- Broadcast **DHCP** request and listen for DHCP response to get an IP address, learn IPs of DNS, next-hop router
- Broadcast **ARP** message to get MAC of the DNS server
- Issue a **DNS** query to get the IP address of the cse.engin.umich.edu server.
 - This DNS query eventually hits the [engin.umich.edu](https://cse.engin.umich.edu) authoritative server, which responds.
- Broadcast **ARP** message to get MAC of the next-hop router.
- Construct a **TCP** connection with the content server by sending a TCP SYN message to port 443.
- Perform a **TLS** handshake to establish a secure connection.
- Send an **HTTP** GET request to get the webpage.

Q2 Spanning Tree

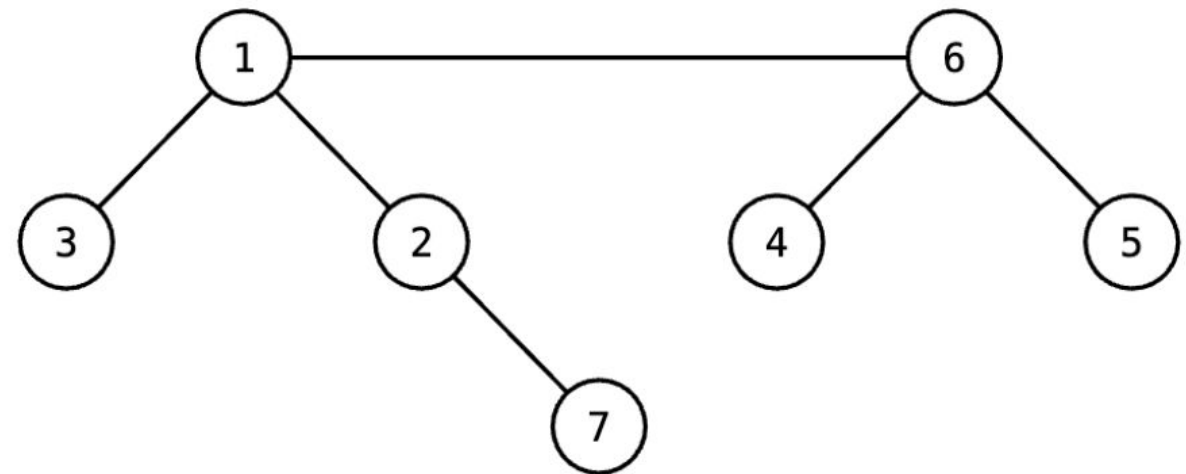
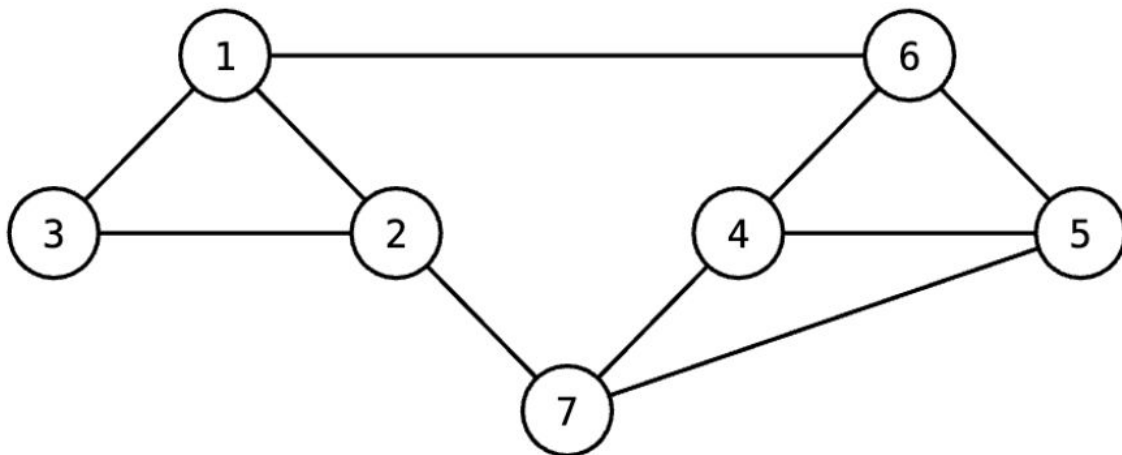
Consider the following graph of nodes in a switched Ethernet network. Suppose these nodes construct a spanning tree among themselves using the algorithm from lecture, electing the node with the smallest identifier as the root. What will be the constructed spanning tree?



Q2 Spanning Tree

Consider the following graph of nodes in a switched Ethernet network. Suppose these nodes construct a spanning tree among themselves using the algorithm from lecture, electing the node with the smallest identifier as the root. What will be the constructed spanning tree?

We do not need to trace through the entire algorithm; the idea here is to only preserve the edges that are on the shortest path from any node to Node 1. This graph has the special property that every node has a unique shortest path to Node 1, so there is no ambiguity in doing so.



Q3 CSMA/CA

Suppose an 802.11b station (wireless) is configured to always reserve the channel with RTS/CTS sequence. How long does it take to transmit a 1000-byte segment of data (from beginning the RTS to receiving the ACK) under the following conditions:

- 10 Mbps transmission rate
- Data-less frame is 32 bytes
- All other stations are idle; no propagation delay or bit errors

Q3 CSMA/CA

Suppose an 802.11b station (wireless) is configured to always reserve the channel with RTS/CTS sequence. How long does it take to transmit a 1000-byte segment of data (from beginning the RTS to receiving the ACK) under the following conditions:

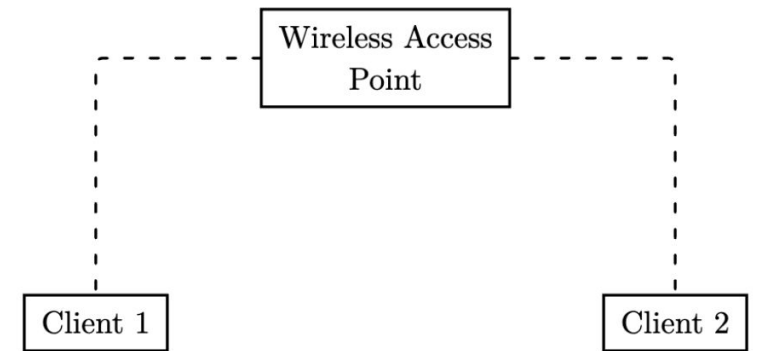
- 10 Mbps transmission rate
- Data-less frame is 32 bytes
- All other stations are idle; no propagation delay or bit errors

$$\left(32 \times \frac{8}{10 \times 10^6} \right) \times 3 + (1000 + 32) \times \frac{8}{10 \times 10^6} = 9.024 \times 10^{-4} \text{ sec} = 902 \text{ usec}$$

Q4 CSMA/CA

Consider the wireless setup shown here. Assume clients use CSMA/CA as a backoff mechanism, and Client 1 and Client 2 are not in direct range of each other.

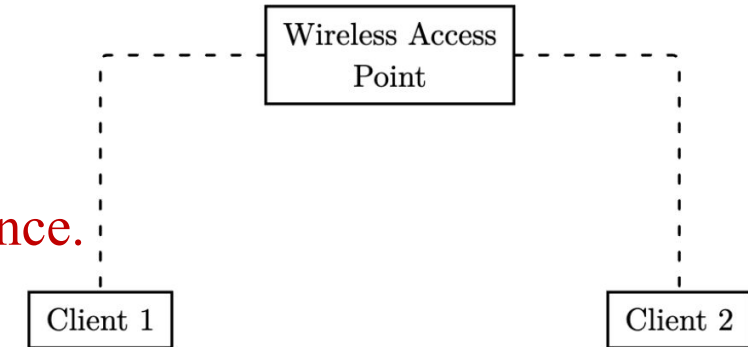
- One-way propagation Client-AP Propagation Delay: 0.5s
- Connection Bandwidth: 16 Megabits/sec
- Timeout before backoff initiated: 1 RTT
- Backoff Time: $2^{(\text{number of times backed off})} + (\text{client \#})$
- Assume negligible transmission delay for network messages



Suppose both Client 1, Client 2 want to send 2 MB of data. Client 1 sends an RTS at $t = 0$ s, while Client 2 sends the initial RTS at $t = 0.5$ s. At what time would Client 2 finally finish sending its data (including the ACK back) to the AP?

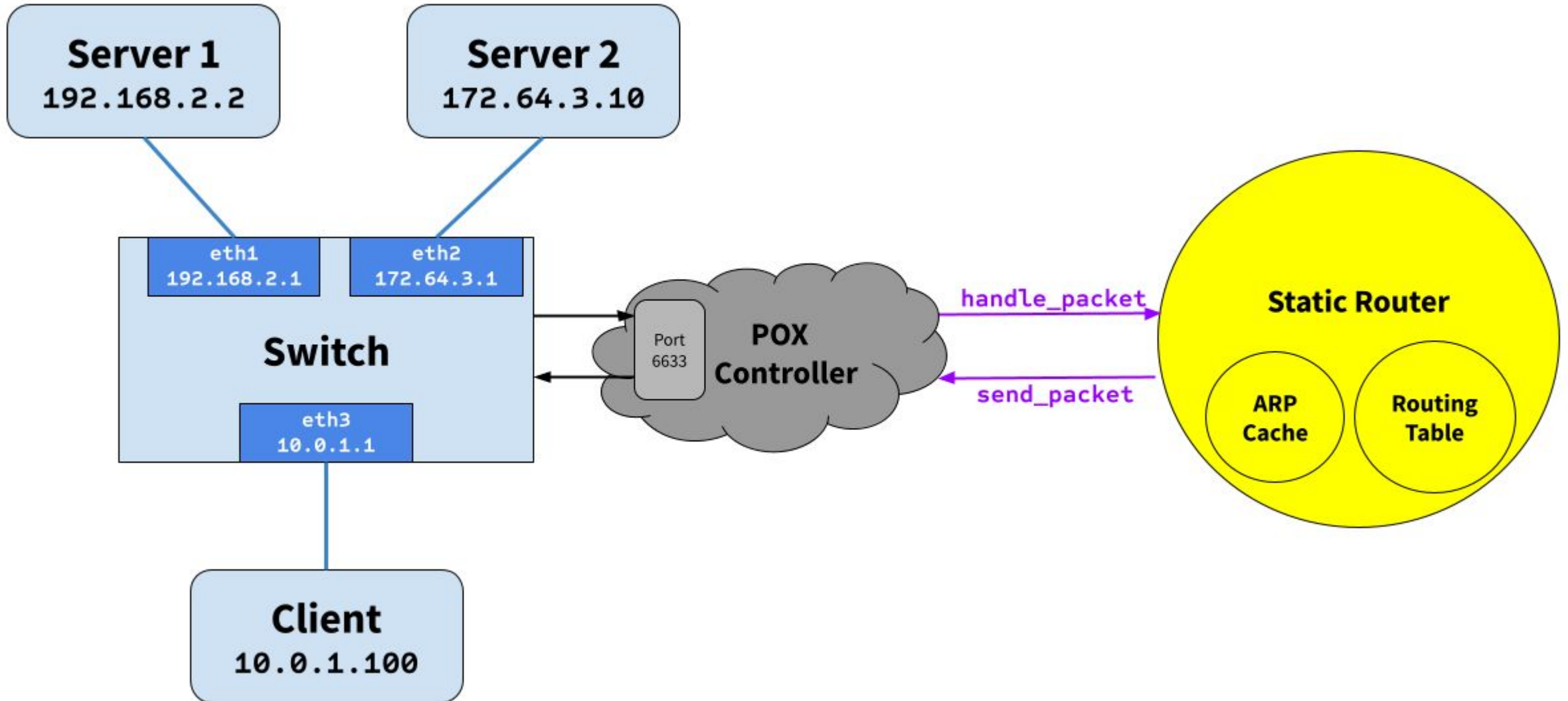
Q4 CSMA/CA

- $t = 0\text{s}$: Client 1 sends RTS
- $t = 0.5\text{s}$: AP broadcasts CTS, Client 2 sends RTS
- $t = 1\text{s}$: Client 1 receives CTS and begins sending data.
Client 2 never receives CTS because the RTS causes interference.
- $t = 1.5\text{s}$: Client 2 begins backoff for $2^0 + 2 = 3\text{s}$
- $t = 2.5\text{s}$: All Client 1 data received at AP
- $t = 3\text{s}$: Client 1 receives ACK.
- $t = 4.5\text{s}$: Client 2 sends RTS
- $t = 5\text{s}$: AP broadcasts CTS
- $t = 5.5\text{s}$: Client 2 begins sending data
- $t = 7\text{s}$: All Client 2 data received at AP
- $t = 7.5\text{s}$: Client 2 receives ACK.

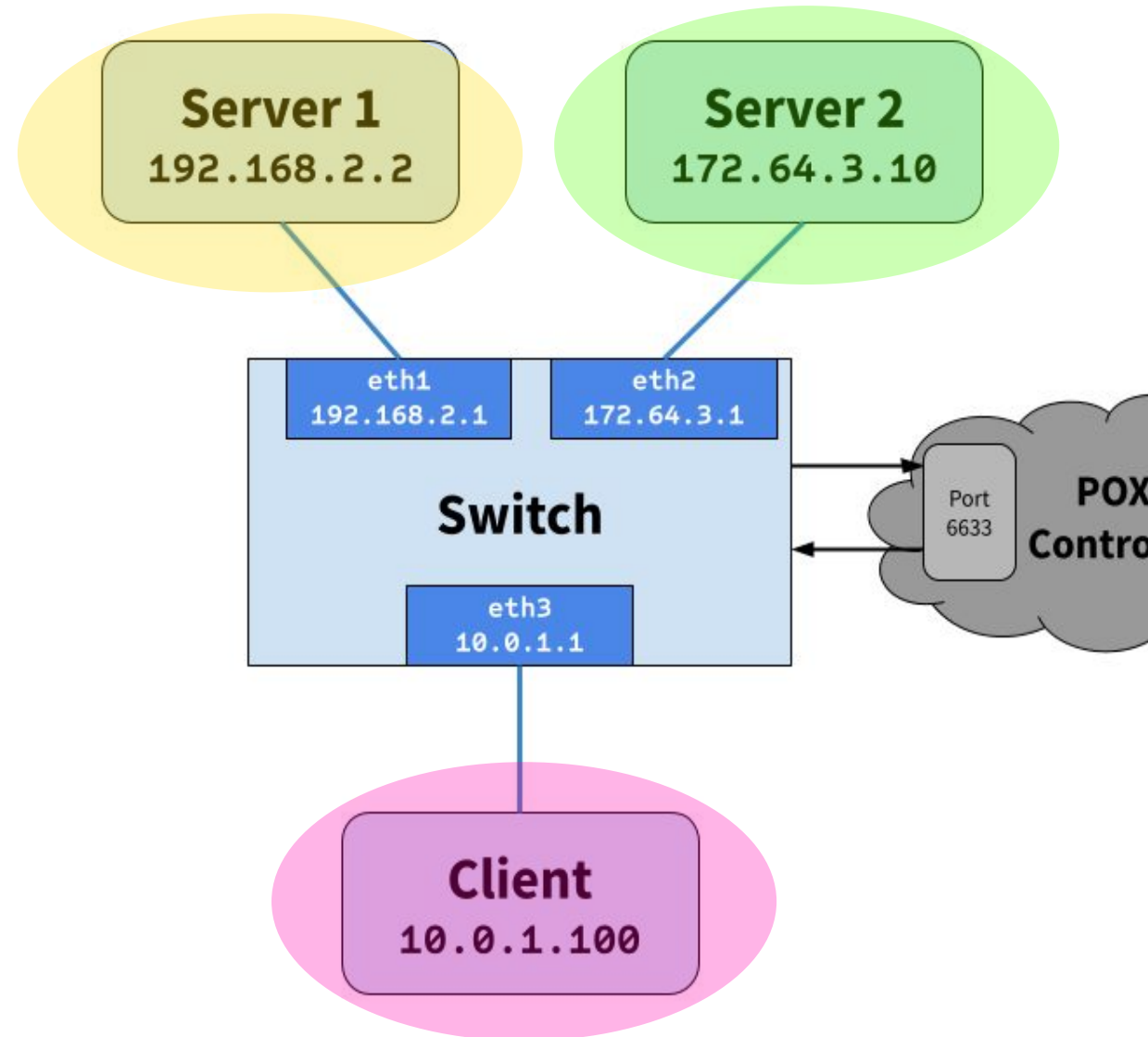


Project 4

Project 4 Architecture



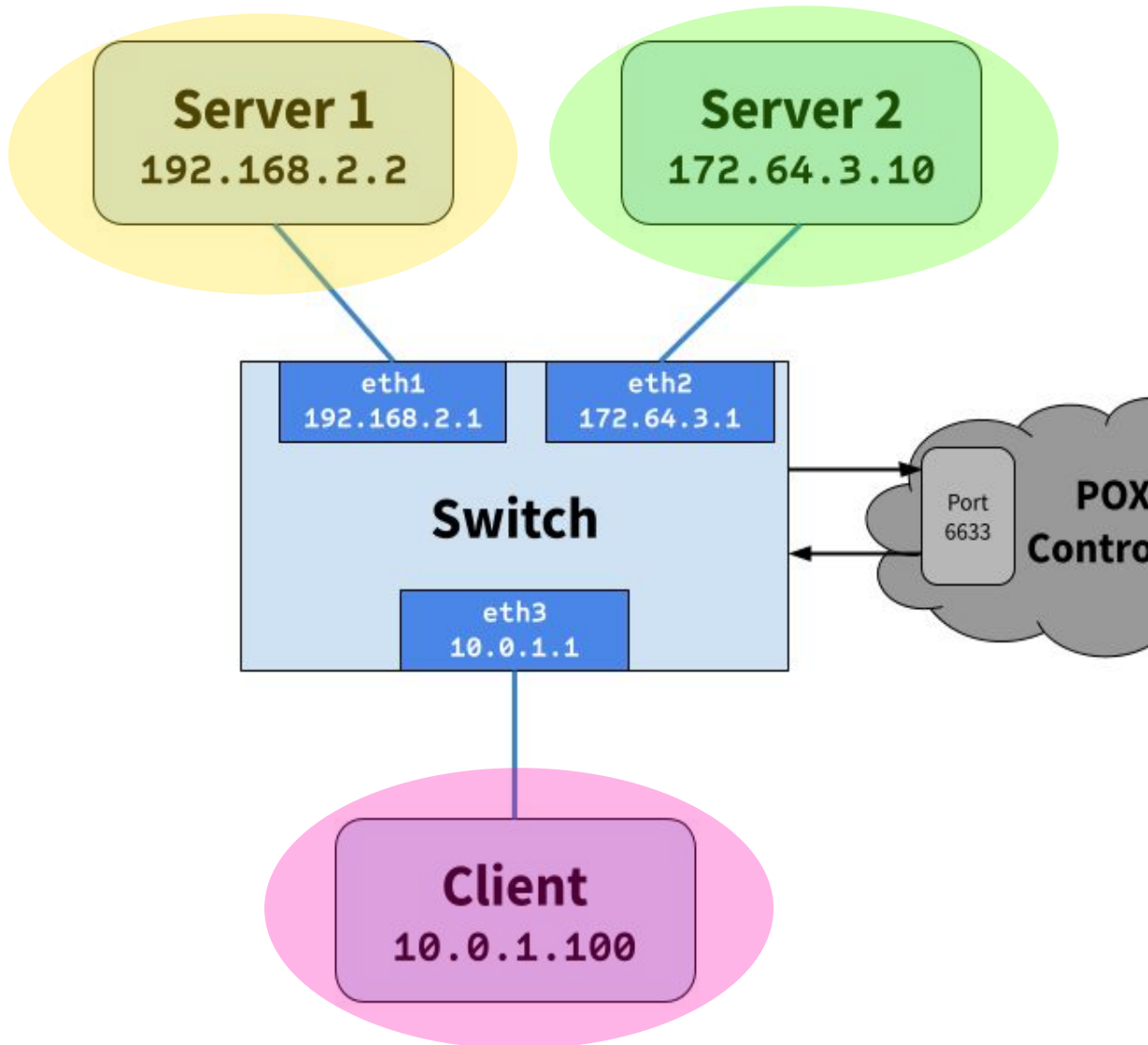
Project 4: Router between LANs



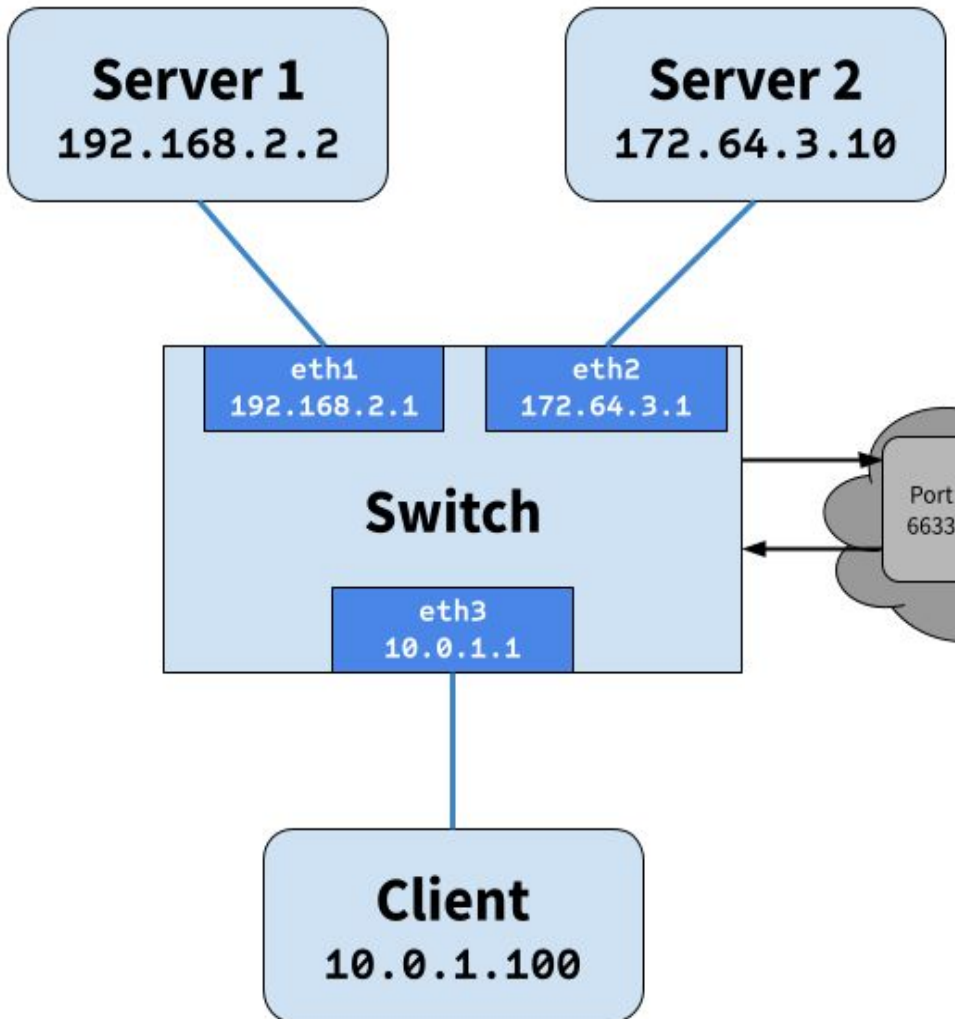
- The switch **bridges LANs**; each interface belongs to a different LAN and has its own MAC, IP.
 - We have only one machine per-LAN to simplify things here, but could have more!
- **Consequence:** Should not forward packets that don't make sense outside the LAN across LANs (i.e. ARP requests).
- **Consequence:** Should replace source MAC with MAC of outgoing interface for forwarded packets.

Project 4 Exercise: Protocols

Client wants to send a TCP message to Server 1. What happens?



Project 4: Protocols



Client wants to send a TCP message to Server 1:

- Client broadcasts ARP request for 10.0.1.1
- **Switch** responds to ARP request with MAC address of eth3
- Client makes IP/TCP packet with dest. 192.168.2.2; sends it to eth3
- **Switch** does a RoutingTable lookup to know that 192.168.2.2 should be sent out of eth1.
- **Switch** queues packet in ARPCache and broadcasts ARP request from eth1 for 192.168.2.2
- Server 1 responds to ARP request with its MAC, causing the **switch** to put this info in its ARP cache
- **Switch** forwards the TCP packet to Server 1, with the source MAC set to eth1 and the dest MAC set to MAC of Server 1.

Project 4 Exercise: LPM

3. Suppose a router uses a **longest prefix match** algorithm for routing. If its routing table has the following entries:

Prefix	Netmask	Next Hop
192.168.0.0	255.255.0.0 (/16)	Router A
192.168.10.0	255.255.255.0 (/24)	Router B
192.168.10.128	255.255.255.128 (/25)	Router C
192.168.10.127	255.255.255.192 (/26)	Router D

Which next hop will an IP packet with a destination IP 192.168.10.130 go to?

Project 4: LPM

3. Suppose a router uses a **longest prefix match** algorithm for routing. If its routing table has the following entries:

Prefix	Netmask	Next Hop
192.168.0.0	255.255.0.0 (/16)	Router A
192.168.10.0	255.255.255.0 (/24)	Router B
192.168.10.128	255.255.255.128 (/25)	Router C
192.168.10.127	255.255.255.192 (/26)	Router D

Which next hop will an IP packet with a destination IP 192.168.10.130 go to?

Project 4: IP Addresses

IP addresses are always represented in **network order** (big endian, high byte to low byte).

IPv4 addresses are 4 8-bit integers, so you might see them represented as `int32` – this is just for convenience. You should really think of them as a **4-element array of `int8`s**.

Takeaway: Don't do order conversions (`ntohl/htonl`) for `RoutingTable`.

C++: Casts and <algorithm>

Casting

C++ has four different types of casts:

- **Static Cast:** Convert between different numeric types, or related pointer types.
- **Dynamic Cast:** Used with polymorphism (convert parent-to-child, for instance).
- **Reinterpret Cast:** Convert anything to anything!
- **Const Cast:** Change whether an expression is const or not. There are some (very few) legitimate use cases for this. It's not quite as bad as using a goto, but it's getting there!

Static Cast

The compiler should be able to guarantee casting at compile-time. OK to lose precision.

```
int a = 10;
double b = static_cast<double>(a);

float c = 5.0;
int d = static_cast<int>(c);

Base *e = new Base();
Derived *f = static_cast<Derived *>(g);

const char *g = "hello";
const void *h = static_cast<const void *>(g);

const char *i = "hello";
const void *j = static_cast<void *>(g); // NOT ALLOWED -- dropping the const

uint32_t *k = new uint32_t(5);
int32_t *l = static_cast<int32_t *>(e); // NOT ALLOWED
```

Reinterpret Cast

Reinterpret any sequence of bits as any other sequence of bits!

Very powerful (and useful, especially for Project 4) – but be careful not to abuse it.

```
vector<char> packet;  
ethernet_packet* eth = reinterpret_cast<ethernet_packet*>(packet.data());  
auto protocol_type: int = eth->ethertype;
```

Can't quite do *anything* – see https://en.cppreference.com/w/cpp/language/reinterpret_cast.

Casting

Never use C-style casts (you don't know which cast is actually being used; hides intent, obfuscates what is actually going on with the code).).

```
1 int iVariable = 20;  
2     float fVariable = (float) iVariable / 10;
```

It's also generally better to stay away from implicit casts, but this isn't as big a deal (since it will only work for very simple static casts anyway).

```
int iVariable = 10;  
    float fVariable = iVariable;
```

Algorithm Library

“The algorithms library defines functions for a variety of purposes (e.g. searching, sorting, counting, manipulating) that operate on ranges of elements” (from <https://en.cppreference.com/w/cpp/algorithm>).

Batch operations

Defined in header `<algorithm>`

<code>for_each</code>	applies a unary function object to elements from a range (function template)
<code>ranges::for_each</code> (C++20)	applies a unary function object to elements from a range (algorithm function object)
<code>for_each_n</code> (C++17)	applies a function object to the first N elements of a sequence (function template)
<code>ranges::for_each_n</code> (C++20)	applies a function object to the first N elements of a sequence (algorithm function object)

Search operations

Defined in header `<algorithm>`

<code>all_of</code> (C++11) <code>any_of</code> (C++11) <code>none_of</code> (C++11)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (function template)
<code>ranges::all_of</code> (C++20) <code>ranges::any_of</code> (C++20) <code>ranges::none_of</code> (C++20)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (algorithm function object)
<code>ranges::contains</code> (C++23) <code>ranges::contains_subrange</code> (C++23)	checks if the range contains the given element or subrange (algorithm function object)
<code>find</code> <code>find_if</code> <code>find_if_not</code> (C++11)	finds the first element satisfying specific criteria (function template)
<code>ranges::find</code> (C++20) <code>ranges::find_if</code> (C++20) <code>ranges::find_if_not</code> (C++20)	finds the first element satisfying specific criteria (algorithm function object)
<code>ranges::find_last</code> (C++23) <code>ranges::find_last_if</code> (C++23) <code>ranges::find_last_if_not</code> (C++23)	finds the last element satisfying specific criteria (algorithm function object)

Fold operations (since C++23)

Defined in header `<algorithm>`

<code>ranges::fold_left</code> (C++23)	left-folds a range of elements (algorithm function object)
<code>ranges::fold_left_first</code> (C++23)	left-folds a range of elements using the first element as an initial value (algorithm function object)
<code>ranges::fold_right</code> (C++23)	right-folds a range of elements (algorithm function object)
<code>ranges::fold_right_last</code> (C++23)	right-folds a range of elements using the last element as an initial value (algorithm function object)
<code>ranges::fold_left_with_iter</code> (C++23)	left-folds a range of elements, and returns a <code>pair</code> (iterator, value) (algorithm function object)
<code>ranges::fold_left_first_with_iter</code> (C++23)	left-folds a range of elements using the first element as an initial value, and returns a <code>pair</code> (iterator, optional) (algorithm function object)

Modifying sequence operations

Copy operations

Defined in header `<algorithm>`

<code>copy</code> <code>copy_if</code> (C++11)	copies a range of elements to a new location (function template)
<code>ranges::copy</code> (C++20) <code>ranges::copy_if</code> (C++20)	copies a range of elements to a new location (algorithm function object)
<code>copy_n</code> (C++11)	copies a number of elements to a new location (function template)
<code>ranges::copy_n</code> (C++20)	copies a number of elements to a new location (algorithm function object)
<code>copy_backward</code>	copies a range of elements in backwards order (function template)
<code>ranges::copy_backward</code> (C++20)	copies a range of elements in backwards order (algorithm function object)
<code>move</code> (C++11)	moves a range of elements to a new location (function template)

Algorithm Library

Why use the library?

- **Speed:** The code in there has been optimized beyond anything you would reasonably do.
- **Correctness:** By writing less boilerplate code, you avoid annoying bugs.
- **Conciseness:** Your code becomes far more concise; you can now write things with much fewer lines (even if each line takes a bit longer to think about and write).
- **Readability:** Other developers can instantly understand the high-level purpose of a function just by looking at which algorithm it is using, rather than having to go through a for-loop and piece together what is going on.

Algorithm Library

```
void remove_dups(std::vector<int> &numbers) {  
    std::sort(first: numbers.begin(), last: numbers.end());  
    // std::unique rearranges elements s.t. unique elements appear at the start  
    auto last: __wrap_iter<int *> = std::unique(first: numbers.begin(), last: numbers.end());  
    numbers.erase(first: last, last: numbers.end());  
}
```

```
vector<int> generate_random_numbers(int n, int low, int high) {  
    std::vector<int> numbers(n);  
  
    std::random_device rd;  
    std::mt19937 gen(sd: rd());  
    std::uniform_int_distribution<> dis(a: 0, b: 100);  
  
    std::generate(first: numbers.begin(), last: numbers.end(), gen: [&]()-> int { return dis(&g: gen); });  
}
```

Algorithm Library

```
void remove_dups(std::vector<int> &numbers) {  
    std::sort(first: numbers.begin(), last: numbers.end());  
    // std::unique rearranges elements s.t. unique elements appear at the start  
    auto last: __wrap_iter<int *> = std::unique(first: numbers.begin(), last: numbers.end());  
    numbers.erase(first: last, last: numbers.end());  
}
```

```
vector<int> generate_random_numbers(int n, int low, int high) {  
    std::vector<int> numbers(n);  
  
    std::random_device rd;  
    std::mt19937 gen(sd: rd());  
    std::uniform_int_distribution<> dis(a: 0, b: 100);  
  
    std::generate(first: numbers.begin(), last: numbers.end(), gen: [&]()-> int { return dis(&g: gen); });  
}
```

Exercise 1: <algorithm>

We are implementing a circular buffer of non-negative integers using a vector. This means that we have a vector of size n such that there is some contiguous range of non-negative integers, while the rest of the vector is -1 s (to represent blank/invalid values).

Given such a vector, create a function that shifts the valid range to start at index 0 instead.

Exercise 2: <algorithm>

We are performing some network measurements of latency. These are stored as a vector of doubles. We want to ensure that we filter out any negative values, as these represent missing data. Once this is done, we want to find out how many high-latency measurements we have, defined as any measurement that is more than 5x the mean latency. Write a function that accomplishes this.

Thanks!